

Gestion de projet



Image Pixabay

Choix méthodologique :

Pour le développement de l'application EcoRide, j'ai fait le choix d'une organisation agile.

Plus concrètement, j'ai appliqué la méthode Kanban avec des User Stories et une planification visuelle via Trello. Cette méthode m'a permis d'avoir une gestion par priorités, de rester flexible face aux imprévus techniques, de livrer régulièrement des versions fonctionnelles et de suivre efficacement l'évolution du projet.

Evolution du projet :

Mon projet s'est articulé autour de différentes phases. Le mot d'ordre était un développement étape par étape :

1. Analyse des besoins : Donc lecture du sujet, découpage fonctionnel en suivant les User Stories et les besoins, et écriture de ces User Stories dans Trello.
2. Conception : Zoning papier, wireframes et mockups sur Figma, charte graphique sur Figma, réflexion UX/UI, création des diagrammes.
3. Environnement de travail : Installation des outils (XAMPP, MongoDB Compass, VSCode, GitHub, Composer...).
4. Développement front end : Routage en Javascript, création des pages HTML/CSS/Bootstrap.
5. Développement back end : Modèle en PHP connecté à MySQL et MongoDB, Contrôleur en Javascript avec de l'injection HTML dynamique. Développement dans

un premier temps page par page en suivant les User Stories associés aux pages quand c'était possible.

6. Finalisation des user Stories : Les User Stories qui interconnecte plusieurs pages ont été développée en dernier.
7. Tests : Les tests ont été menée sur chaque fonction écrite et/ou chaque page finalisée. Des comptes variés de profils test ont été utilisées à différentes fin. Une fois l'application fini, un test sur son ensemble a été mené. Suite à ces teste, des corrections et nettoyages ont été menés.
8. Déploiement : Synchronisation de GitHub avec Heroku, vérification des .env, tests en conditions réelles.
9. Livraison finale : Constitution du documentation technique, manuel d'utilisation, dossier de gestion de projet, charte graphique, dépôt GitHub, base de données, maquettes et Trello.

Tout au long du projet, j'ai également tenu un dossier de suivi au format Word. Ce document regroupe mes réflexions, mes idées, des notions de cours et de code que je souhaitais me remémorer ainsi que l'ensemble du code rédigé étape par étape, accompagné des modifications successives (à la manière d'un système de version comme Git). Il m'a servi de support personnel, à la fois rassurant et utile, notamment pour retrouver une idée, retracer l'évolution d'un module ou revisiter une notion technique

Trello :

La plateforme Trello a été mon point pivot pour la gestion convenable du projet. Je pouvais retrouver chaque tâche effectuer, basculer les cartes dans différentes zones et les faire revenir dans la colonne « Révision du code » en cas de bug.

Mon Trello, construit sous forme Kanban, se présentait ainsi (de gauche à droite) :

- Backlog : Contient toutes les idées, tâches futures et User Stories à traiter.
- Conception : C'est un pense-bête pour les principes à respecter (écologie, cohérence graphique, contraintes du sujet).
- A faire : Tâches prêtes à être développées, écrites le plus souvent en User Stories du type : "En tant que, je veux, afin de".

- En cours : Cartes du Kanban actuellement en développement.
- Révision du code : Fonctionnalités terminées nécessitant des corrections, un refactoring ou une relecture de code.
- Test : Fonctionnalités à tester (données dynamiques, affichage, rôle, correction effectuée à tester, responsive...).
- Terminé (branche développement) : Fonctionnalités prêtes à être livrées mais nécessite une dernière relecture ou une revue de code avant le merge sur la branche stable.
- Terminé (merge branche main) : Fonctionnalités prêtes et validées, intégrées dans la branche principale.
- Conception terminé : Liste les étapes de design finalisées (zoning, wireframes, mockups, charte graphique, logo).
- Environnement de travail installé : Colonne dédiée à la mise en place technique : IDE, extensions, serveur local, accès MongoDB/MySQL, installation Composer, etc.

Précision sur des étapes du développement :

Environnement de travail :

J'ai commencé par installer les outils nécessaires pour un environnement de travail confortable et fonctionnel :

- XAMPP (Apache + MySQL),
- MongoDB Compass pour les logs NoSQL,
- VSCode avec les extensions utiles (Git Graph, MongoDB for VSCode, PHP Intelephense, PHP Namespace Resolver, PHP Unit, PHP Server, Live Sass Compiler),
- Git pour le versionnage,
- Composer pour les dépendances PHP (phpmailer, dotenv, mongodb).

Chaque outil a été validé et suivi dans la colonne dédiée de Trello (carte : « Installer les extensions VSCode » dans la colonne « Environnement de travail installé »).

Phase de conception :

Avant le codage, j'ai réfléchi à la structure globale :

- J'ai réalisé un zoning sur papier puis les wireframes et mockups sur Figma,
- Construction d'une charte graphique écoresponsable avec des couleurs naturelles mais plus sur un ton foncé pour réduire l'utilisation de la batterie. J'ai également utilisé la police intégrée de Bootstrap qui évite de surcharger le code et permet de garder une police populaire ("*Segoe UI*", *Roboto*, "*Helvetica Neue*", *Arial*, sans-serif).
- Rédaction de User Stories centrées sur les besoins des utilisateurs :

"En tant que chauffeur, je souhaite pouvoir appuyer sur le bouton « démarrer » depuis mon espace afin de lancer le covoiturage. "

- Création d'un diagramme de classe via dbdiagram.io,
- Élaboration de diagrammes d'utilisation et de séquence via draw.io pour les parcours

Mise en place du routage JavaScript :

Lors de la conception du projet, j'ai fait le choix d'utiliser PHP vanilla et non un framework comme Symfony ou Laravel. Ce choix était autorisé et je voyais là-dedans, dans le cadre de ma formation, l'occasion d'apprendre l'ensemble des bases de PHP (structure de projet, routage, gestion des formulaires, sessions, sécurité...). Ainsi, j'ai pu mieux comprendre le fonctionnement d'un framework qui automatise certains aspects lors de la programmation.

Cela me permettait d'avoir également plus de contrôle sur le projet et PHP vanilla est plus léger qu'un framework, le projet contenant uniquement ce dont il a besoin.

Dans ce cadre, j'ai développé mon propre système de routage inspiré d'un exercice de travaux pratique abordé dans un cours.

J'ai construit une page socle « index.php » sur lequel se greffe un système de navigation dynamique.

Le routeur identifie les éléments à injecter selon l'attribut id de la balise <main>, ce qui permet de charger les pages de manière fluide sans rechargement complet.

Ce système m'a permis d'implémenter une gestion des droits d'accès et des redirections conditionnelles selon le rôle de l'utilisateur (visiteur, passager, chauffeur, etc.).

Enfin, il a grandement facilité la structuration du code selon le modèle MVC, en séparant clairement les responsabilités entre la Vue, le Modèle et le Contrôleur.

Développement des Vues principales :

J'ai ensuite conçu les pages principales (accueil, inscription, espace personnel, covoiturage...) en HTML/CSS avec Bootstrap 5. La structure est responsive par défaut grâce au système de grille de Bootstrap et l'intégration respecte la charte graphique grâce à une palette de couleurs personnalisée définie en SCSS.

Intégration dynamique grâce au Contrôleur et au Modèle :

Une fois les vues terminées, j'ai repris les User Stories de ma plateforme Trello, qui sont étiquetées par page, afin de développer le contrôleur et le modèle associés, page par page.

Je me suis concentré sur les User Stories n'impliquant pas de logique d'interaction complexe entre plusieurs pages.

Chaque développement suivait un cycle complet avant de passer à la fonctionnalité suivante. Ce cycle était constitué de la création des fichiers JavaScript pour le contrôleur, l'envoi des requêtes fetch vers les fichiers PHP du modèle, la gestion sécurisée des données en MySQL via PDO, et l'enregistrement des logs ou actions spécifiques dans MongoDB pour plus de flexibilité.

Les User Stories plus complexes :

J'ai terminé les User Stories par le développement de ceux impliquant une interconnexion entre plusieurs pages.

Ces fonctionnalités, plus complexes, nécessitaient une coordination entre différentes vues, contrôleurs et modèles. Elles ont donc été finalisées en dernier, une fois que chaque composant isolé était fonctionnel, afin d'assurer une intégration fluide, facile et cohérente dans l'ensemble de l'application.

Les derniers rôles et pages à avoir été développés ont été les employés puis l'administrateur.

Tests :

Pour tester chaque fonctionnalité, j'ai créé des profils de test différents (Passager, Chauffeur, Passager-chauffeur avec des comptes Gmail associés et enfin un compte employé et administrateur).

Par ces profils, j'ai notamment pu tester :

- L'envoi d'emails par PHPMailer avec SMTP sécurisé,
- Vérifier le bon affichage et la bonne récupération des logs MongoDB (avis, suspensions, crédits) même si je pouvais les visualiser dans MongoDB Compass,

- Vérifier également le bon changement et récupération des données sur MySQL, même si je pouvais visualiser ces modifications sur phpMyAdmin,
- Vérifier les limitations d'accès pour chaque rôle.

Déploiement continu avec Git et Heroku :

J'ai utilisé pour ce projet un workflow Git structuré :

- Une branche « main » qui est la branche principale et contient la version stable,
- Une branche « dev » qui contenait la branche en développement.
- Des branches « feature/* » et « bugfix/* » en sous branche de « dev » pour développer des fonctionnalités spécifiques ou réparer des bugs. Je faisais ensuite un merge avec la branche dev, une fois le test effectué.

La base relationnelle est hébergée via JawsDB MySQL et la base NoSQL via MongoDB Atlas.

Les variables d'environnement sont stockées dans .env pour la version locale et inscrit dans .gitignore pour ne jamais être poussée sur le repository. Dans la version déployée, les variables d'environnement sont inscrites dans les « Settings » d'Heroku .

Bilan :

Grâce à cette méthode je pouvais me repérer plus aisément dans mon projet, ce qui m'a facilité la tâche pour la rédaction du README.md.

Avoir tenu un journal de bord m'a également rassuré et permit d'approfondir mes connaissances d'une manière plus sereine,

J'ai pu prioriser facilement les tâches grâce à Trello et visualiser rapidement mon parcours me permettant ainsi d'anticiper ma gestion du temps.

Ensuite :

Même si l'application est déployée, coté développement j'ai encore des tâches affichées dans mon trello. Ce sont des tâches en dehors des User Stories mais que j'ai rajouté parce qu'elles me semblent nécessaire pour assurer le bon fonctionnement et l'optimisation de l'application.

Voici plusieurs d'entre elle avec des propositions de solution :

« En tant que Passager, suite à une réservation de plusieurs places pour un trajets, je souhaite laisser un avis pour l'ensemble de mes réservations. »

Solution possible : Ajouter une colonne nb_places_reservees dans la collection MongoDB avis_trajet ou dans la table reservations pour indiquer combien de places concernent l'avis. Cela permettrait à l'utilisateur de laisser un seul avis par trajet, indépendamment du nombre de places réservées.

« En tant qu'Employé, lors de la validation d'un avis concernant plusieurs réservations pour un trajet, je souhaite que les crédits soient attribués en équivalence avec le nombre de places réservés. »

Solution possible : Lors de la validation de l'avis par l'employé, le script PHP gestion_avis.php devra inclure une récupération du nombre de places réservées et appliquer le crédit en fonction de ce nombre. Exemple : prix * nb_places.

« En tant qu'Employé, je souhaite voir sur un tableau l'ensemble des avis refusé ou validé. »

Solution possible : Ajouter une interface avec des filtres (statut = validé/refusé) et interroger MongoDB pour afficher tous les avis selon leur statut.

« Lors de la création d'un compte, il faudrait que la note initiale soit à 0 et non à NULL. »

Solutions possibles : Ajouter une valeur par défaut (0) dans la base de données ou l'ajouter explicitement dans le script inscription.php.

« En tant qu'Utilisateur, je souhaite un bouton pour supprimer mon compte. »

Solution possible : Ajouter un bouton "Supprimer mon compte" dans l'espace utilisateur, qui envoie une requête vers un fichier supprimer_compte.php. Celui-ci pourra :

- désactiver le compte (note = -1) ou
- supprimer toutes les données associées après confirmation de l'utilisateur.

« En tant que Passager, je souhaite être remboursé entièrement si mon trajet est annulé »

Solution possible : Lorsque le trajet passe à l'état annulé, un script pourrait récupérer tous les passagers concernés, recrediter leur compte (selon le prix * nb de places), et enregistrer un log MongoDB remboursement_annulation.

« En tant qu'Utilisateur, je souhaite avoir une page « Erreur 404 » plus user friendly. »

Solution possible : Créer une page 404.html avec un visuel (icône, illustration SVG), un message clair ("Oups ! Page introuvable") et un bouton de retour vers l'accueil.

« En tant qu'Utilisateur, je souhaite pouvoir recevoir par mail une réinitialisation du mot de passe sur la page de connexion en cas d'oubli. »

Solution possible : Ajouter un lien « Mot de passe oublié ? » sur la page de connexion, qui redirige vers un formulaire demandant l'adresse email de l'utilisateur.

À la soumission un token unique est généré et enregistré avec une date d'expiration et un mail est envoyé avec un lien.

« En tant qu'Utilisateur, je souhaite pouvoir avoir un visuel convivial et plus fonctionnel sur le smartphone. »

Solution possible : Intégration responsive via Bootstrap 5 (grille col-12 col-md-6, classes d-flex, text-center, etc.), design mobile-first avec @media personnalisés dans app.scss. Les composants sont adaptés pour le tactile (boutons larges, textes lisibles, espacement augmenté).

« En tant qu'Utilisateur, je souhaite pouvoir recharger mon compte en crédit. »

Solution possible : Ajouter une page "Rechargement" avec choix du montant et paiement sécurisé (type Stripe). Une fois validé, incrémenter les crédits dans la base MySQL et enregistrer un log de rechargement dans MongoDB (logs_rechargement).

« En tant qu'Utilisateur chauffeur, je ne dois pas avoir accès à la page « Covoiturage »

Solution possible : Dans allRoutes.js, j'attribue les droits d'accès par page et dans navbar.js je peux cacher l'onglet « Covoiturage » si l'utilisateur qui se connecte est un chauffeur.

« En tant qu'Utilisateur, je veux autoriser l'utilisation de cookies lors de la connexion. »

Solution possible : Suite à la connexion, afficher un message javascript invitant à valider l'autorisation de cookies.

« En tant qu'Utilisateur, je veux que le site soit optimisé pour les malvoyants. »

Solution possible : Vérifier et rajouter la balise « alt » avec un descriptif pour chaque image.

« En tant qu'Utilisateur, je veux pouvoir effectuer une recherche depuis la page d'accueil.»

Solution possible : Connecter le formulaire de la page d'accueil avec covoit.js et suite à une recherche effectuer une redirection vers la page Covoiturages.